INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
KULLIYYAH OF ENGINEERING

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

EXPERIMENT 1:

DIGITAL LOGIC SYSTEM: BASIC LOGIC GATES, ELECTRONIC CIRCUIT INTERFACING, BASIC ALU, 7-SEGMENT DISPLAY, ICS BASED INTERFACING APPLICATIONS

SECTION 1

GROUP 1

NO	NAME	MATRIC NUMBER	PROGRAMME
1	'AISYAH EL-'ILLIYYINA BINTI MOHD FAUZI	2319982	BMCT
2	AFIQ ASRAF BIN MOHD EZUAN'IZAM	2410541	BMCT
3	AHMAD AIMAN BIN ABDUL HALIM	2417921	BMCT
4	AFIQ HUSAINI BIN MOHAMAD IZANI	2411757	BMCT

DATE OF SUBMISSION: 12 OCTOBER 2026

ABSTRACT

This experiment aimed to interface a common-cathode seven-segment display and two tactile pushbuttons with an Arduino Uno microcontroller to construct an interactive user-controlled decimal counter. Seven digital GPIO pins (D2–D8) were wired to drive segments A–G through individual 220 Ω series current-limiting resistors, restricting per-segment current to approximately 13.64 mA to prevent thermal overload. Concurrently, the increment (D12) and reset (D13) pushbuttons were connected as active-LOW digital inputs utilizing 10 kΩ external pull-up resistors. Digital pins D0 and D1 were intentionally unassigned to prevent hardware conflicts with the USB serial UART communications bus during sketch uploads. Initial circuit performance was validated through Tinkercad simulation prior to physical breadboard implementation. The embedded C++ control firmware continually polled input states, detected falling-edge button triggers, and executed a 50 ms software debouncing delay to filter mechanical contact chatter. Upon actuation, the program either incremented the numerical count from 0 to 9 — wrapping back to 0 via modulo arithmetic — or instantly restored the display to 0. Numerical rendering was governed by a 10-digit Boolean segment activation truth table. Empirical testing confirmed accurate sequential visual representation across all ten digits without logic faults. Primary technical challenges included mechanical switch bouncing, pin-mapping errors, and loose breadboard contacts. For future iterations, implementing non-blocking timer-based debouncing routines and adopting I²C display driver ICs (such as the PCF8574) or shift registers is recommended to optimize firmware responsiveness and conserve microcontroller GPIO pins.

TABLE OF CONTENT

ABSTRACT	**2**
TABLE OF CONTENT	**3**
1.0. INTRODUCTION AND THEORY	**4**
1.1. MATERIALS AND EQUIPMENTS	**5**
1.2. EXPERIMENTAL SETUP	**6**
1.3. METHODOLOGY AND PROGRAMMING LOGIC	**7**
1.4. DATA COLLECTION AND ANALYSIS	**12**
1.5. RESULTS AND DISCUSSION	**14**
1.6. CONCLUSION	**17**
RECOMMENDATIONS	**18**
REFERENCES	**19**
APPENDICES	**20**
ACKNOWLEDGEMENT	**21**
STUDENT'S DECLARATION	**22**

1.0. INTRODUCTION AND THEORY

Digital systems process binary information using discrete electrical voltage levels representing Logic 1 (HIGH, 5 V) and Logic 0 (LOW, 0 V) to transfer data and control electronic output hardware. Microcontrollers incorporate an internal Arithmetic Logic Unit (ALU) to execute mathematical accumulation, incrementing routines, and logical state evaluations.

Digital display control requires evaluating Boolean truth conditions to translate numerical values into specific LED illumination patterns. A standard 7-segment display consists of seven rectangular Light Emitting Diodes (segments A, B, C, D, E, F, G) and an optional decimal point (dp) arranged in a figure-eight configuration. In a common cathode configuration, the cathode (negative) terminals of all individual segment LEDs are internally connected to a shared ground node . To illuminate any specific segment, the Arduino Uno microcontroller must set the corresponding digital General Purpose Input/Output (GPIO) pin to an active-HIGH logic output (5 V). Directly connecting a microcontroller's digital pin to an LED anode creates a low-resistance path that draws excessive current, risking permanent thermal damage to both the LED segment and the ATmega328P microcontroller's output driver. To prevent overcurrent, a 220 Ω current-limiting resistor is wired in series between each microcontroller output pin and its respective display segment anode (A-G).

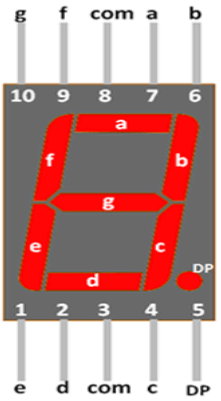
Microcontrollers utilize an internal Arithmetic Logic Unit (ALU) to execute binary operations, such as incrementing counter variables (count = count + 1) or evaluating conditional reset routines. To manually control counter state changes, tactile pushbuttons are interfaced with digital input pins. When the button is pushed, the signal will be received by the Arduino Uno, and the count on the 7-segment display can be controlled. To ensure that the microcontroller's input pin rests at a stable, defined logic level (HIGH) when no other component is actively driving the circuit, two 10 kΩ pull-up resistors are connected between a signal pin and the positive power supply voltage (VCC).

This experiment aims to interface an Arduino Uno with a common cathode 7-segment display and two pushbuttons using 220 Ω current-limiting resistors and 10 kΩ pull-up resistors to ensure safe current levels. The software objectives are to execute an Arduino C++ sketch mapping Boolean truth tables across digital output pins, incorporate software debouncing routines, sequentially increment displayed digits from 0 to 9 via the increment button, and reset the output to 0 via the reset button.

It is hypothesized that outputting active-HIGH logic signals (5 V) from output pins through 220 Ω resistors will complete the circuit path to ground on a common cathode 7-segment display, safely illuminating individual LED segments according to active Boolean output matrices. Furthermore, wiring pushbuttons to input pins with 10 kΩ pull-up resistors will maintain a default HIGH state that drops to active-LOW (0 V) upon actuation, allowing the firmware state machine to reliably increment the display count from 0 to 9 or clear it back to 0.

1.1. MATERIALS AND EQUIPMENTS

1. Arduino Uno board
2. Common Cathode 7-Segment Display:
    - Segment A (7) → Pin D03
    - Segment B (6) → Pin D02
    - Segment C (4) → Pin D08
    - Segment D (2) → Pin D07
    - Segment E (1) → Pin D06
    - Segment F (9) → Pin D04
    - Segment G (10) → Pin D05
    - Segment DP (5) → Pin D09
    - Common Cathode Pin (3) → Pin Power GND



3. 4-pin pushbuttons (2 of them):
    - Pushbutton 1 (Increment Button) → Pin D12
    - Pushbutton 2 (Reset Button) → Pin D13
4. 220 Ω current-limiting resistors (8 of them) connected in series between the digital output pins and the 7-segment display's pins.
5. 10 kΩ pull-upresistors (2 of them) connected in series between the digital input pins and the pushbuttons' pins.
6. Breadboard
7. Male-to-male jumper wires
8. Personal Computer with Arduino IDE (C++ compiler and serial monitor)

1.2. EXPERIMENTAL SETUP

Circuit Wiring Instructions (Figure 1.1):

1. Power & Ground Rails:
    - **Arduino GND Pin** → Breadboard Negative Power Rail (−)
2. 7-Segment Display
    - **Display Pin 8 (Common Cathode)** → Breadboard Ground Rail (−)
    - **Arduino Pin D03** → 220 Ω Resistor → Display Pin 7 (**Segment A**)
    - **Arduino Pin D02** → 220 Ω Resistor → Display Pin 6 (**Segment B**)
    - **Arduino Pin D08** → 220 Ω Resistor → Display Pin 4 (**Segment C**).
    - **Arduino Pin D07** → 220 Ω Resistor → Display Pin 2 (**Segment D**)
    - **Arduino Pin D06** → 220 Ω Resistor → Display Pin 1 (**Segment E**)
    - **Arduino Pin D04** → 220 Ω Resistor → Display Pin 9 (**Segment F**)
    - **Arduino Pin D05** → 220 Ω Resistor → Display Pin 10 (**Segment G**)
    - **Arduino Pin D09** → 220 Ω Resistor → Display Pin 5 (**Segment DP**)
3. Increment Pushbutton Circuit (Active-LOW)
    - **Pushbutton 1 Terminal 1 → Arduino Digital Pin D12**
    - **Pushbutton 1 Terminal 1**→ 10 kΩ Pull-Up Resistor → Arduino 5V Pin
    - **Pushbutton 1 Terminal 2** → Breadboard Ground Rail (-)
4. Increment Pushbutton Circuit (Active-LOW)
    - **Pushbutton 2 Terminal 1 → Arduino Digital Pin D13**
    - **Pushbutton 2 Terminal 1**→ 10 kΩ Pull-Up Resistor → Arduino 5V Pin
    - **Pushbutton 2 Terminal 2** → Breadboard Ground Rail (-)

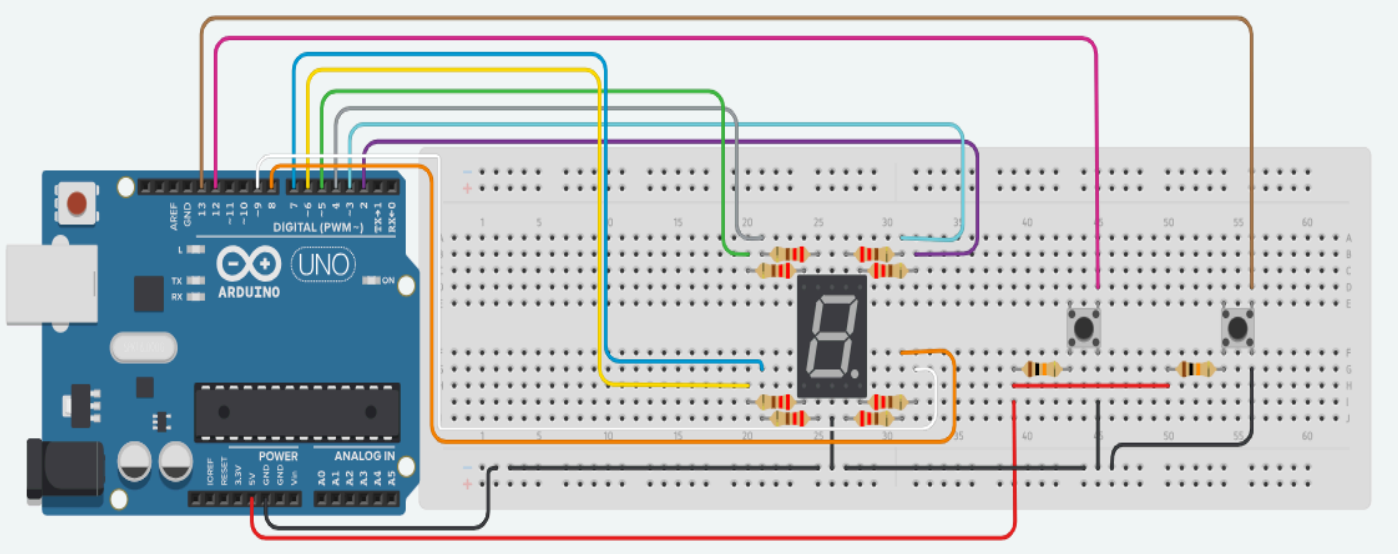


Figure 1.1

1.3. METHODOLOGY AND PROGRAMMING LOGIC

Procedural Steps:

1. Assemble the hardware components on the breadboard following the setup in Section 1.2.
2. Connect the Arduino Uno to the host PC using a USB cable and launch the Arduino IDE.
3. Write, compile, and upload the C++ firmware sketch to the board.
4. Open the Serial Monitor in the Arduino IDE to observe system status.
5. Press the Increment Pushbutton (D12) sequentially to step the counter from 0 to 9.
6. Press the Reset Pushbutton (D13) to return the display counter state to 0.

Logic Flow Diagrams (Figure 1.2):

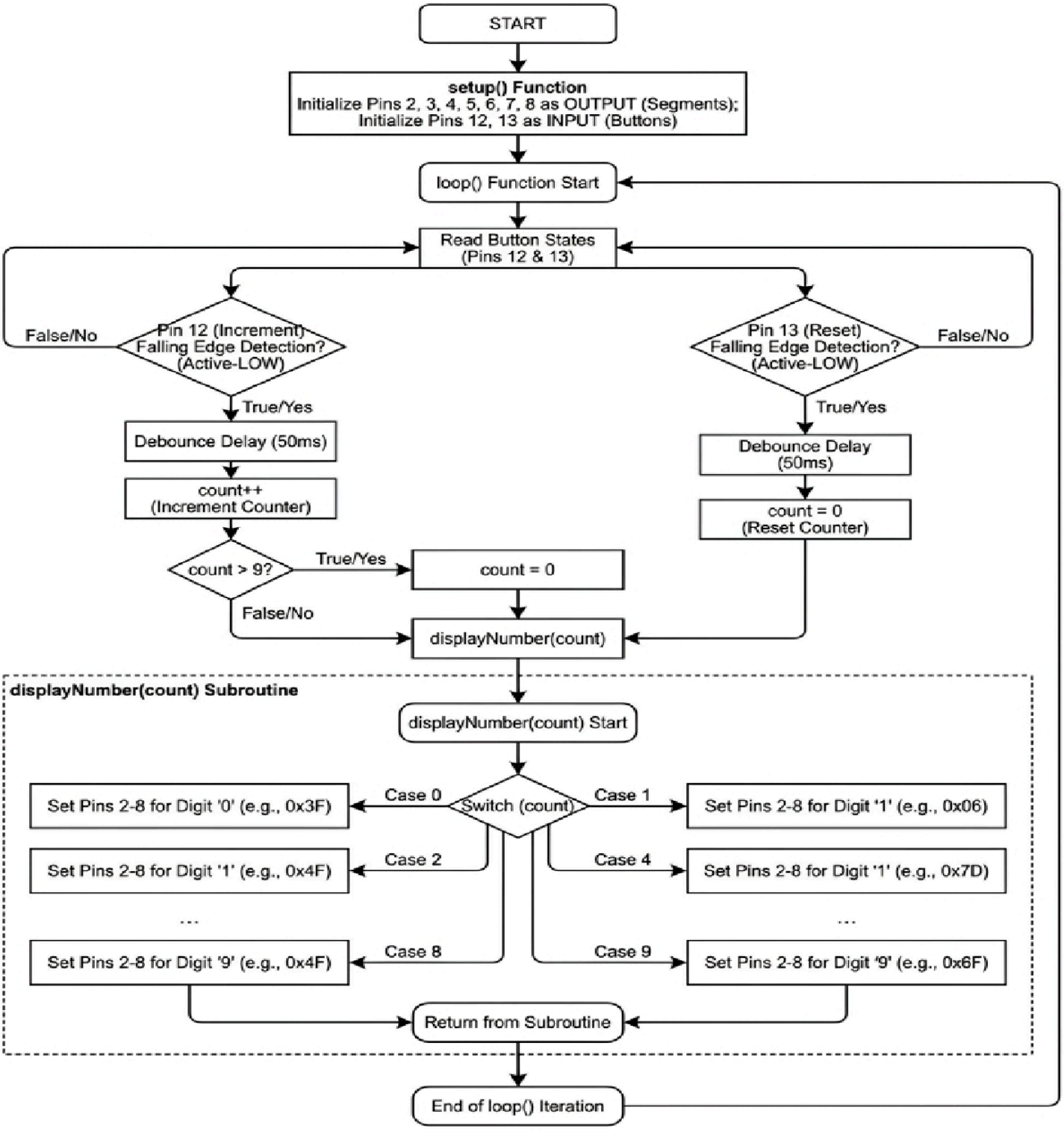


Figure 1.2

Complete Arduino C++ Control Firmware using Arduino IDE:

```
// Define the pins for each segment based on your configuration
const int segmentA = 3;
const int segmentB = 2;
const int segmentC = 8;
const int segmentD = 7;
const int segmentE = 6;
const int segmentF = 4;
const int segmentG = 5;

// Define the pushbutton pins
const int incrementBtn = 12;  //[cite: 3]
const int resetBtn = 13;     //[cite: 3]

// Variables for state tracking
int count = 0;
int lastIncState = HIGH;
int lastResState = HIGH;

void setup() {
  pinMode(segmentA, OUTPUT);
  pinMode(segmentB, OUTPUT);
  pinMode(segmentC, OUTPUT);
  pinMode(segmentD, OUTPUT);
  pinMode(segmentE, OUTPUT);
  pinMode(segmentF, OUTPUT);
  pinMode(segmentG, OUTPUT);

  pinMode(incrementBtn, INPUT);
  pinMode(resetBtn, INPUT);
}

void loop() {
  int currentIncState = digitalRead(incrementBtn);
  int currentResState = digitalRead(resetBtn);

  // Increment button logic
  if (currentIncState == LOW && lastIncState == HIGH) {
    count++;
    if (count > 9) {
      count = 0; // Sequence resets after 9[cite: 3]
    }
```

```
    delay(50); // Debounce
  }

  // Reset button logic
  if (currentResState == LOW && lastResState == HIGH) {
    count = 0;
    delay(50); // Debounce
  }

  lastIncState = currentIncState;
  lastResState = currentResState;

  // Update the 7-segment display based on the current count
  displayNumber(count);
}

// Function to handle the segment logic for each number
void displayNumber(int num) {
  // First, turn off all segments
  digitalWrite(segmentA, LOW); digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW); digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW); digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);

  // Then, turn on the specific segments for the current number
  switch (num) {
    case 0: // A,B,C,D,E,F
      digitalWrite(segmentA, HIGH); digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, HIGH); digitalWrite(segmentD, HIGH);
      digitalWrite(segmentE, HIGH); digitalWrite(segmentF, HIGH);
      break;
    case 1: // B,C
      digitalWrite(segmentB, HIGH); digitalWrite(segmentC, HIGH);
      break;
    case 2: // A,B,G,E,D
      digitalWrite(segmentA, HIGH); digitalWrite(segmentB, HIGH);
      digitalWrite(segmentG, HIGH); digitalWrite(segmentE, HIGH);
      digitalWrite(segmentD, HIGH);
      break;
    case 3: // A,B,C,D,G
      digitalWrite(segmentA, HIGH); digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, HIGH); digitalWrite(segmentD, HIGH);
      digitalWrite(segmentG, HIGH);
      break;
```

```
    case 4: // B,C,F,G
      digitalWrite(segmentB, HIGH); digitalWrite(segmentC, HIGH);
      digitalWrite(segmentF, HIGH); digitalWrite(segmentG, HIGH);
      break;
    case 5: // A,C,D,F,G
      digitalWrite(segmentA, HIGH); digitalWrite(segmentC, HIGH);
      digitalWrite(segmentD, HIGH); digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, HIGH);
      break;
    case 6: // A,C,D,E,F,G
      digitalWrite(segmentA, HIGH); digitalWrite(segmentC, HIGH);
      digitalWrite(segmentD, HIGH); digitalWrite(segmentE, HIGH);
      digitalWrite(segmentF, HIGH); digitalWrite(segmentG, HIGH);
      break;
    case 7: // A,B,C
      digitalWrite(segmentA, HIGH); digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, HIGH);
      break;
    case 8: // A,B,C,D,E,F,G
      digitalWrite(segmentA, HIGH); digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, HIGH); digitalWrite(segmentD, HIGH);
      digitalWrite(segmentE, HIGH); digitalWrite(segmentF, HIGH);
      digitalWrite(segmentG, HIGH);
      break;
    case 9: // A,B,C,D,F,G
      digitalWrite(segmentA, HIGH); digitalWrite(segmentB, HIGH);
      digitalWrite(segmentC, HIGH); digitalWrite(segmentD, HIGH);
      digitalWrite(segmentF, HIGH); digitalWrite(segmentG, HIGH);
      break;
  }
}
```

1.4. DATA COLLECTION AND ANALYSIS

1. **Digital Input Switch Condition-Response:** Since the 10 kΩ pull-up resistors have been connected in series with the Arduino 5V pin and the pushbuttons' pins respectively, the pushbuttons will act as active-LOW components. When the argument is TRUE, the components will send LOW (0 V) to the microcontroller component. Table 1.1 details the physical switch states, input voltages, logic levels, and program execution triggers.

Input Switch	**Physical State (Switch**	**Input Voltage (V_{in})**	**Logic State**	**Counter State Machine Trigger**	**System Response**
Increment (D12)	Unpressed (Open)	5.0 V DC	1 (HIGH)	Idle State	Display maintains current numerical count (count).
Increment (D12)	Pressed (Closed)	0.0 V DC	0 (LOW)	Falling-edge Transition	Counter increments (count++). If count > 9, resets to 0.
Reset (D13)	Unpressed (Open)	5.0 V DC	1 (HIGH)	Idle State	Display maintains current numerical count (count).
Reset (D13)	Pressed (Closed)	0.0 V DC	0 (LOW)	Falling-edge Transition	Counter instantly clears to 0 (count = 0).

Table 1.1

2. **Digital Output Segment:** The common cathode 7-segment display requires discrete active-HIGH logic combinations to illuminate numerical digits 0 through 9. Table 1.2 delineates the complete output logic mapping from Arduino digital pins to display segment anodes, along with active segment combinations and hardware pin logic states.

Displayed Digit	**Logic States / Output Mapping**							**Active Segments**
	Seg. A (D03)	**Seg. B (D02)**	**Seg. C (D08)**	**Seg. D (D07)**	**Seg. E (D06)**	**Seg. F (D04)**	**Seg. G (D05)**	
0	1	1	1	1	1	1	0	A,B,C,D,E,F
1	0	1	1	0	0	0	0	B,C
2	1	1	0	1	1	0	1	A,B,D,E,G
3	1	1	1	1	0	0	1	A,B,C,D,G
4	0	1	1	0	0	1	1	B,C,F,G

5	1	0	1	1	0	1	1	A,C,D,F,G
6	1	0	1	1	1	1	1	A,C,D,E,F,G
7	1	1	1	0	0	0	0	A,B,C
8	1	1	1	1	1	1	1	A,B,C,D,E,F,G
9	1	1	1	1	0	1	1	A,B,C,D,F,G

Table 1.2

1.5. RESULTS AND DISCUSSION

The Digital Logic System experiment was designed to demonstrate the application of digital logic principles through the interfacing of an Arduino Uno with a common-cathode 7-segment display and pushbuttons. The main objective was to control the numerical output displayed on the 7-segment display using digital signals generated by the Arduino.

The 7-segment display consists of seven individual LED segments, labelled A to G, which can be controlled independently to produce different numerical patterns. Each segment is connected to a digital output pin of the Arduino through a 220 Ω current-limiting resistor. The common-cathode terminals are connected to ground, allowing the corresponding LED segments to illuminate when a HIGH logic signal is applied.

According to the programmed operating principle, different combinations of HIGH (1) and LOW (0) signals determine which segments are illuminated. This demonstrates how binary logic states can be converted into a visible numerical output.

The experiment also involves two pushbuttons, namely the increment and reset buttons. The increment button is intended to increase the displayed number sequentially from 0 to 9, while the reset button returns the displayed value to 0.

Figure F.1: Physical hardware setup showing the Arduino Uno connected to the 7-segment display.

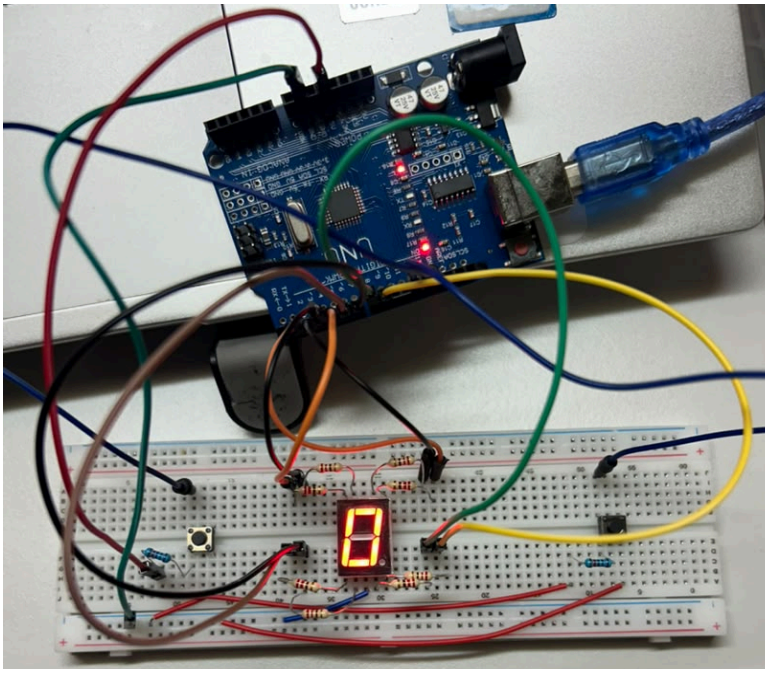
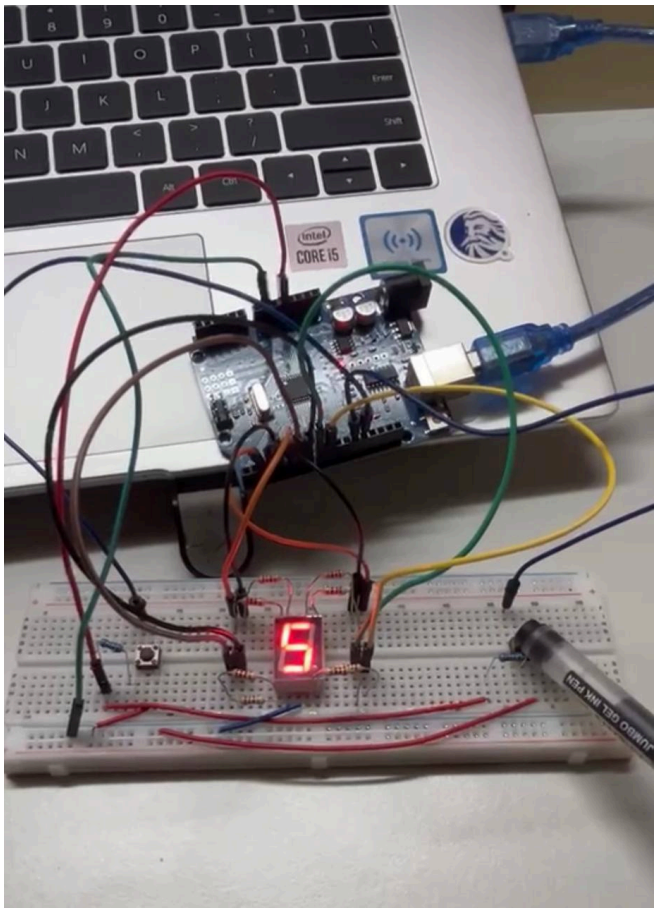


Figure F.2: Numerical output displayed on the 7-segment display.



Discussion of Errors and Technical Limitations

Several hardware and software factors may influence the performance of the digital logic system.

1. Incorrect wiring and loose connections

Improper connections between the Arduino, breadboard, resistors, and 7-segment display may cause certain LED segments to remain off or illuminate incorrectly. A loose jumper wire may also interrupt the electrical connection, resulting in an incomplete numerical pattern. This issue can be reduced by checking the circuit connections against the pin diagram and ensuring that all components are securely connected.

2. Incorrect segment pin configuration

Each LED segment must correspond to the correct digital pin defined in the Arduino program. If the physical wiring does not match the programmed pin assignments, the displayed number may be incorrect. Therefore, the Arduino pin configuration should be verified before uploading and executing the program.

Questions:

How can you interface an I²C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED.

To interface an I²C LCD with an Arduino Uno, the LCD module is connected using an I²C I/O expander backpack, which reduces the physical wiring to just four pins: VCC connected to 5V, GND to ground, SDA (Serial Data) to analog pin A4, and SCL (Serial Clock) to analog pin A5. The Arduino operates as the master controller on a two-wire serial bus, sending packetized data bytes and commands to the slave driver IC at a designated hardware address. This physical setup eliminates the need for 6 to 11 parallel GPIO pins required by standard LCDs, significantly simplifying hardware assembly on the breadboard.

The I²C LCD is easier to interface with Arduino because it requires fewer pins and uses library functions to display information. The 7-segment display requires direct control of individual LED segments, while the matrix LED requires row-column control to produce patterns. Therefore, the I²C LCD is more suitable for displaying text and detailed information, whereas 7-segment and matrix LED displays are useful for numerical outputs and visual patterns.

From a software perspective, the coding principles differ significantly across the three display architectures. The 7-segment display relies on low-level parallel GPIO control, in which the microcontroller CPU executes explicit **digitalWrite()** commands to set individual pin states to HIGH or LOW based on a manually coded Boolean truth table. In contrast, the I²C LCD leverages high-level software abstraction through specialized hardware libraries (**Wire.h** and **LiquidCrystal_I2C.h**), allowing developers to output complete alphanumeric strings using simple commands, while offloading character rendering to the onboard controller. Meanwhile, a matrix LED typically utilizes dedicated driver ICs to manage synchronous row-column multiplexing and register shift operations, updating 8-byte frame buffers to render complex patterns or animations without overburdening the Arduino's processor or pin capacity.

1.6. CONCLUSION

In conclusion, the experiment successfully interfaced a common-cathode seven-segment display and two pushbuttons with an Arduino Uno to build a simple decimal counter. The increment button increased the count from 0 to 9 and wrapped back to 0, while the reset button returned the count to 0, meeting all the stated objectives. All ten digits were displayed correctly, matching both the truth table in Table 1.2 and the segment patterns in the code. The hardware and software worked together as expected: the 10 kΩ pull-up resistors kept the button inputs at a stable HIGH state, so each press produced a falling edge that the program could detect, and the 50 ms debounce delay helped reduce false counts. The 220 Ω resistors limited the current to each segment, protecting both the display and the Arduino. Although establishing the correct wiring was challenging, the problems were resolved through repeated checking and testing of the circuit. The main takeaways are that correct wiring, current-limiting resistors and stable input states are essential for reliable digital interfacing. In addition, the seven-segment display requires direct control of each segment, whereas an I2C LCD needs fewer pins and is better suited for displaying text and detailed information.

RECOMMENDATIONS

For future experiments, it is recommended that the circuit connections be checked carefully before powering on the Arduino Uno. The wiring of the 7-segment display, push buttons, resistors, and ground connections should be arranged according to the circuit diagram provided. Each connection should be verified to ensure that the components are connected to the correct pins and that the circuit functions as intended.

Furthermore, it is encouraged to troubleshoot wiring problems systematically rather than repeatedly reconnecting components without checking the circuit configuration. If the circuit does not operate as expected, the jumper wires, resistor connections, and component orientation should be inspected before making further changes. Referring to laboratory guidelines and discussing possible solutions with peers can also help identify and resolve connection errors more efficiently.

In addition, it is recommended that the circuit be tested step by step, starting with the individual components before testing the complete system. This approach can help identify errors more easily and reduce troubleshooting time. By following these recommendations, future experiments can be conducted more efficiently, thereby improving the general understanding of electronic circuit interfacing, digital logic, and Arduino programming.

REFERENCES

1. Floyd, Thomas L. (2015). *Digital Fundamentals, 11th Edition* (11). Pearson Education Ltd.

2. Monk, P, S, A, S. (2016). Practical electronics for inventors (Fourth edition). New York: McGraw Hill Education.

3. Monk, S. (2016). Programming Arduino™ Getting Started with Sketches. McGraw-Hill Education.

4. Mano, M., & Ciletti, M. D. Digital Design: With an Introduction to the Verilog HDL, VHDL, and System Verilog.

5. Prabhu, A. (2026, January 20). In-Depth: How Seven Segment Display Works & Interface with Arduino. Last Minute Engineers. https://lastminuteengineers.com/seven-segment-arduino-tutorial/

APPENDICES

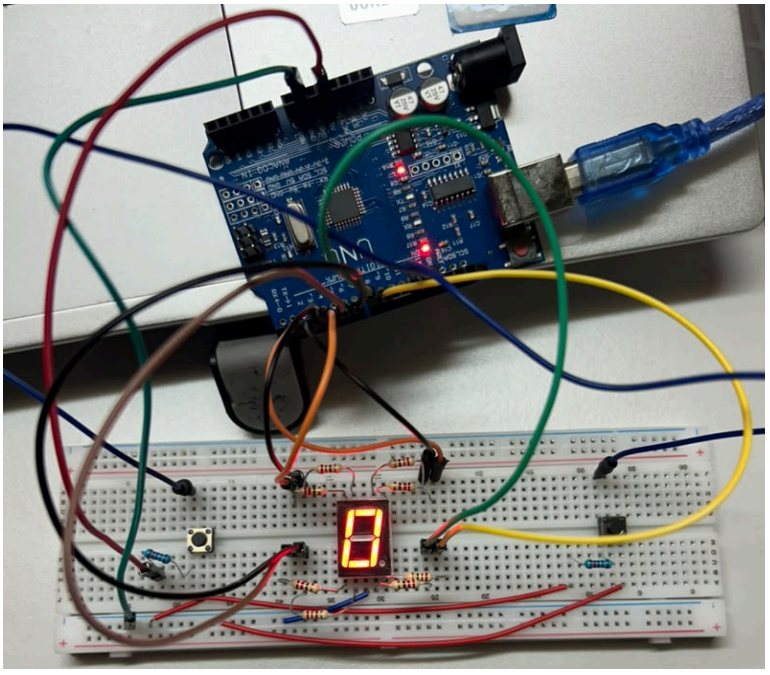


Figure 1.3 (The circuit connections and Pre-testing)

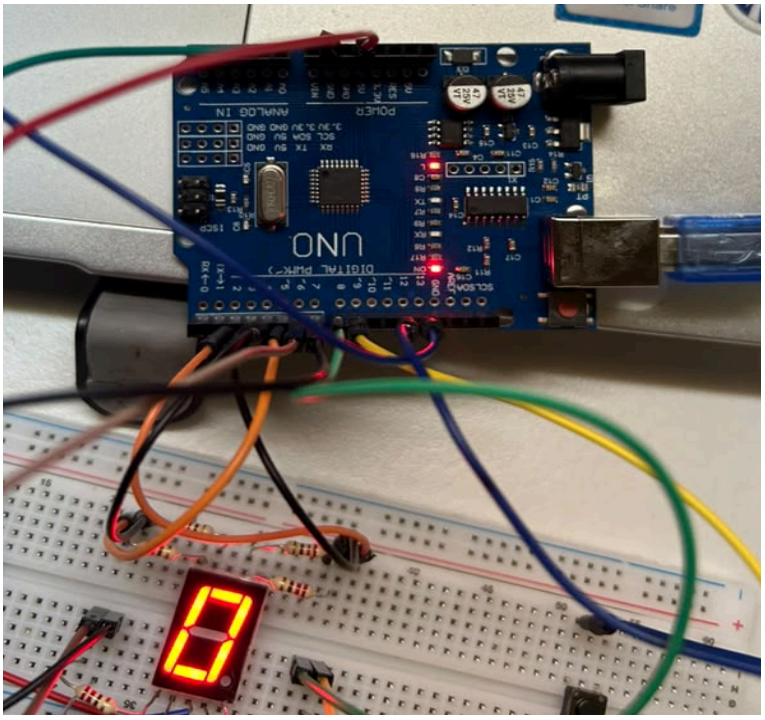


Figure 1.4 (The Arduino Uno's Pins Connection)

The link for the testing video:

https://youtube.com/shorts/19eVswmcC_o?feature=share

ACKNOWLEDGEMENT

First and foremost, we would like to express our gratitude to our lecturer, Dr. Ahmad Imran, an Assistant Professor in the Department of Mechatronics Engineering, and our instructor for this experiment, for providing guidance, instructions, and valuable explanations throughout this Digital Logic System experiment, as well as actively supervising our laboratory session until the end of the class. The guidelines provided were helpful in assisting us in understanding the circuit connections and overcoming the difficulties encountered during the experiment.

We would also like to extend our sincere appreciation to the Department of Mechatronics Engineering, International Islamic University Malaysia (IIUM), for providing the experimental venue, complete materials and equipment, essential guidelines, and supportive laboratory staff who assisted us throughout our practical activities.

STUDENT'S DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report.** The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking,** and this **final printed report** has been **verified by us**.

Name: AHMAD AIMAN BIN ABDUL HALIM Matric No.: 2417921 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: AFIQ ASRAF BIN MOHD EZUAN'IZAM Matric No.: 2410541 Signature:	Read	✓
	Understand	✓
	Agree	✓

Name: ‘AISYAH EL-’ILLIYYINA BT MOHD FAUZI Matric No.: 2319982 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: AFIQ HUSAINI BIN MOHAMAD IZANI Matric No.: 2411757 Signature: 	Read	✓
	Understand	✓
	Agree	✓